Enterprise IT Support Agentic RAG Copilot

Project Architecture and Deployment Runbook

FastAPI | LangGraph | Pinecone | Groq | Tavily | Docker | AWS Lightsail

Operational reference for setup, testing, deployment, verification, and cleanup

Document purpose

This runbook explains how the Enterprise IT Support Agentic RAG Copilot is structured, how a user question moves through the system, how the application is configured and tested, and how the Docker image is deployed to AWS Lightsail. It is intended for project demonstration, handover, troubleshooting, and repeatable redeployment.

Project outcome

The completed solution is a deployable internal IT support assistant. It searches the private company knowledge base first, evaluates evidence quality, uses web search only when necessary, produces a grounded response, and exposes the decision trace through the API and user interface.

Scope

Area	Included
Product	Enterprise IT support copilot for policies, troubleshooting, VPN, MFA, password and software questions
Architecture	Frontend, FastAPI API, LangGraph workflow, Pinecone, Groq, Tavily and audit logging
Operations	Environment setup, ingestion, Docker build, local validation and Lightsail deployment
Security	Environment variables, admin key, private KB priority and secret handling
Evidence	Deployment screenshots and successful Active deployment verification

Contents

1. Product overview
2. Architecture
3. Technology stack
4. Repository structure
5. Agent workflow
6. Local setup and knowledge ingestion
7. Docker packaging and validation
8. AWS Lightsail deployment
9. Verification and operations
10. Troubleshooting and cleanup

1 Product overview

The project addresses repetitive internal IT support requests. Employees can ask questions in natural language rather than searching multiple documents. The system prefers trusted private company content and clearly distinguishes external information when it must use web search.

2 Solution architecture

The system is organized as a request pipeline. The browser sends a question to FastAPI. FastAPI invokes the LangGraph workflow. The workflow retrieves private evidence from Pinecone, grades it, optionally calls Tavily, and uses the Groq-hosted language model to generate the final answer.

```
Employee → HTML/CSS/JavaScript UI → FastAPI → LangGraph
                                        ├── Pinecone private knowledge base
                                        ├── Tavily web fallback
                                        └── Groq LLM → grounded answer
```

Component	Role	Data exchanged
Web UI	Collects questions and displays answer, citations and trace	JSON requests and responses
FastAPI	Validates requests and exposes API routes	Question, answer, citations, upload metadata
LangGraph	Controls routing, retrieval, grading, retries and generation	Shared AgentState
Pinecone	Stores and retrieves private document vectors	Embeddings and document chunks
Groq	Routes, grades, rewrites and generates answers	Prompts and structured responses
Tavily	Provides external web evidence when KB is weak	Search results and citations
SQLite audit	Records question, source and trace	Audit rows

2.1 Request lifecycle

A normal IT question follows Route → Retrieve → Grade → Generate. If private evidence is weak, the workflow searches the web, grades web evidence, and either generates an externally sourced answer, rewrites the query and retries, or stops with an insufficient-evidence response.

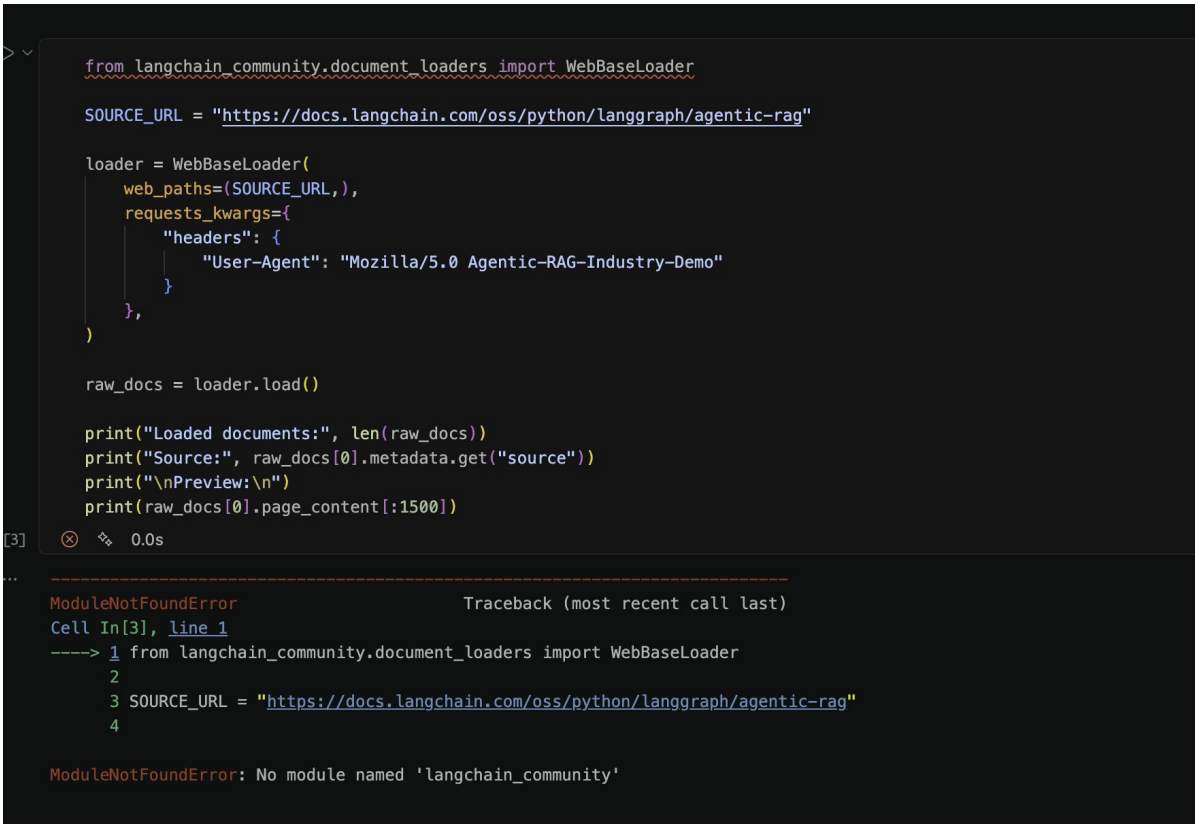


Figure 1. Local project and development reference screenshot

3 Technology stack

Layer	Technology	Purpose
Workflow orchestration	LangGraph	Stateful graph and conditional routing
Language model	Groq through OpenAI-compatible ChatOpenAI	Routing, evidence grading, query rewrite and answer generation
Embeddings	HuggingFace all-MiniLM-L6-v2	Local 384-dimensional document/query vectors
Vector database	Pinecone	Private company knowledge retrieval
Web search	Tavily	External fallback for current or missing information
Backend	FastAPI and Uvicorn	REST API, validation and service runtime
Frontend	HTML, CSS and JavaScript	Employee-facing chat interface
Audit	SQLite	Question, source and execution trace logging
Packaging	Docker	Portable and repeatable runtime
Cloud deployment	AWS Lightsail Containers	Public container hosting

4 Repository structure

```
app/
  api/routes.py          Chat, health and ingestion endpoints
  core/config.py         Centralized settings and environment loading
  rag/state.py            Shared LangGraph state and structured decisions
  rag/vectorstore.py     Embeddings, Pinecone and retriever
  rag/workflow.py        Agentic RAG graph
  services/ingestion.py  File loading and chunking
  services/audit.py      SQLite audit logging
  main.py                FastAPI application
data/sample_kb/          Company IT documents
templates/index.html     Web page
static/css, static/js    UI styling and browser logic
ingest_sample_kb.py      Ingestion script
requirements.txt         Python dependencies
Dockerfile               Container build instructions
run.py                   Local application entry point
```

5 Core API endpoints

Endpoint	Method	Purpose
/api/health	GET	Service health check
/api/chat	POST	Answer an IT support question
/api/ingest	POST	Authorized document upload and indexing
/docs	GET	Interactive FastAPI Swagger interface

6 Agentic RAG workflow

The workflow is agentic because it does not blindly retrieve and generate. Each node makes a controlled decision based on the current state. Structured outputs are used for routing and evidence grades so the graph can follow deterministic branches.

Stage	Operation	Possible result
1. Route	Classify the message as KB or direct conversation	kb or direct
2. Retrieve	Query Pinecone with the current query	Retrieved document chunks
3. Grade KB	Check whether private evidence is sufficient	good or weak
4. Web fallback	Search Tavily when KB evidence is weak	Web evidence and citations
5. Grade web	Check relevance and sufficiency	good or weak
6. Rewrite	Improve vague or weak query	Updated query and retry count
7. Generate	Answer only from selected evidence	Private KB, web, direct or insufficient

Evidence and trust model

Private company content is the preferred source for company policy. Web evidence is used only as a fallback and the generated answer is instructed to warn the user that external information may require IT validation. If neither source is reliable after the configured retry limit, the system returns a safe insufficient-evidence response.

Ingestion lifecycle

```
File → loader → document objects → recursive chunks → embeddings → Pinecone namespace
Question → query embedding → nearest chunks → evidence grade → grounded answer
```

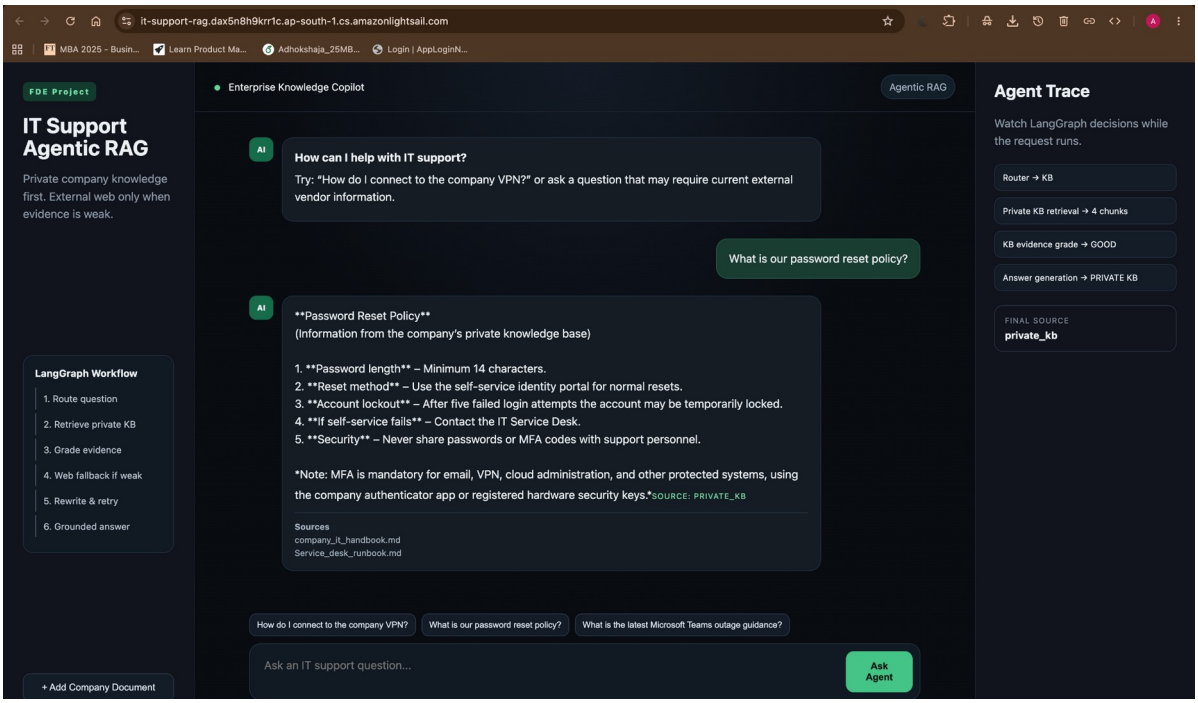


Figure 2. Lightsail local-machine deployment guidance

7 Local setup

The local workflow is used to install dependencies, configure secrets, ingest sample knowledge and verify the API before any cloud deployment.

7.1 Create environment and install packages

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

7.2 Configure environment variables

Create `.env` in the project root. Keep this file local and excluded from Git. In cloud deployment, enter the same variables manually in the Lightsail deployment configuration.

Variable	Purpose
OPENAI_API_KEY	Provider key used by the OpenAI-compatible client configuration
OPENAI_MODEL	Configured chat model identifier
TAVILY_API_KEY	External search fallback
PINECONE_API_KEY	Vector database access
PINECONE_INDEX_NAME	Pinecone index
PINECONE_NAMESPACE	Private document namespace
EMBEDDING_MODEL	Embedding model identifier
TOP_K	Number of chunks retrieved
MAX_RETRIES	Maximum query rewrite retries
ADMIN_API_KEY	Protects document ingestion

7.3 Ingest the sample knowledge base

```
python ingest_sample_kb.py
```

7.4 Run locally

```
python run.py
# Browser: http://127.0.0.1:8000
# Swagger: http://127.0.0.1:8000/docs
```

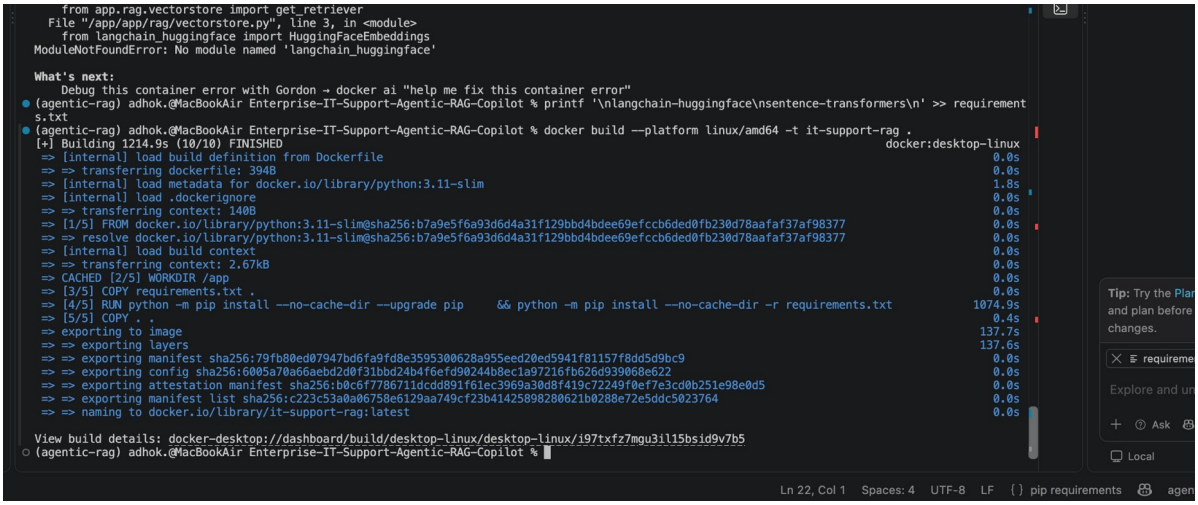


Figure 3. Local Docker and project setup reference

8 Docker packaging and validation

Docker packages the application, Python runtime and dependencies into one reproducible image. The image was built for `linux/amd64`, matching the target deployment environment.

8.1 Build image

```
docker build --platform linux/amd64 -t it-support-rag .
```

8.2 Run locally with environment file

```
docker run --rm --platform linux/amd64 \
  -p 8080:8080 \
  --env-file .env \
  it-support-rag
```

8.3 Validate

```
http://localhost:8080/api/health
```

Expected response

```
{"status":"ok","service":"Enterprise IT Support Agentic RAG Copilot"}
```

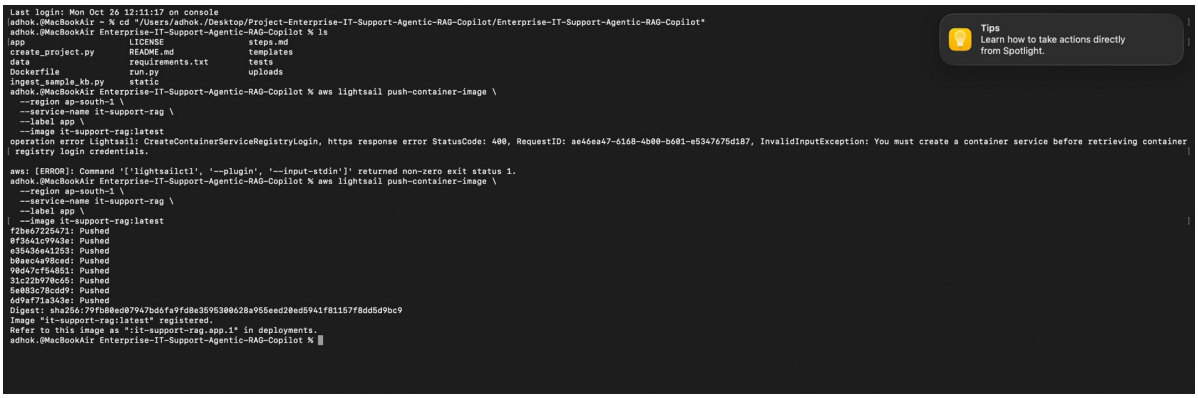


Figure 4. Successful Lightsail image push and generated image reference

Operational note

If a Python module is missing, add it to `requirements.txt`, rebuild the image, push it with a new label, and deploy the newly generated Lightsail image reference.

9 AWS Lightsail deployment

The cloud deployment uses Lightsail Containers. The service was created in the Mumbai region (`ap-south-1`) with one Medium node. The deployment exposes the FastAPI container publicly on HTTP port 8080.

9.1 Create the service

In Lightsail, create a container service named `it-support-rag`, choose the region and capacity, and wait until the service is ready. The service must exist before the image upload command can retrieve registry credentials.

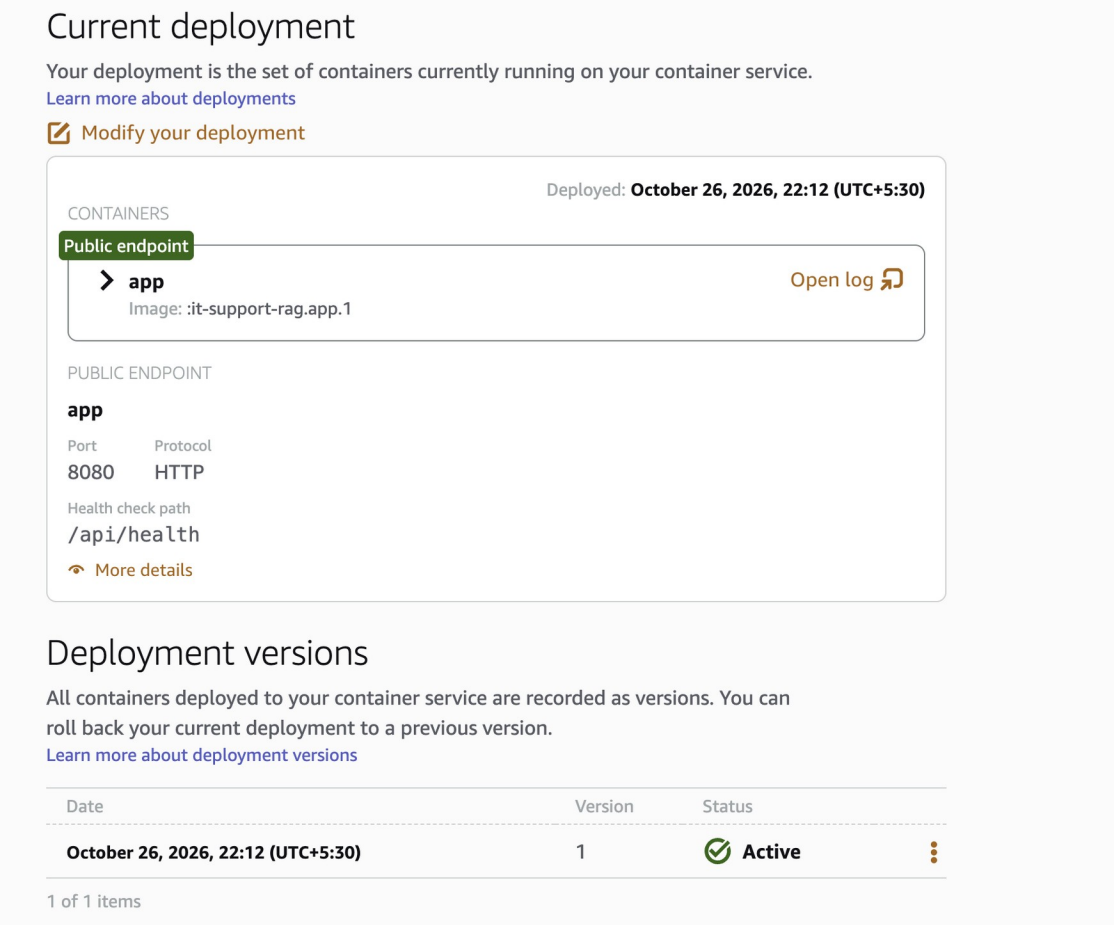


Figure 5. Lightsail container service and deployment configuration

9.2 Push the image

```
aws lightsail push-container-image \
  --region ap-south-1 \
  --service-name it-support-rag \
  --label app \
  --image it-support-rag:latest
```

The command returns a deployment image reference such as `:it-support-rag.app.1`. Use the exact value returned by AWS.

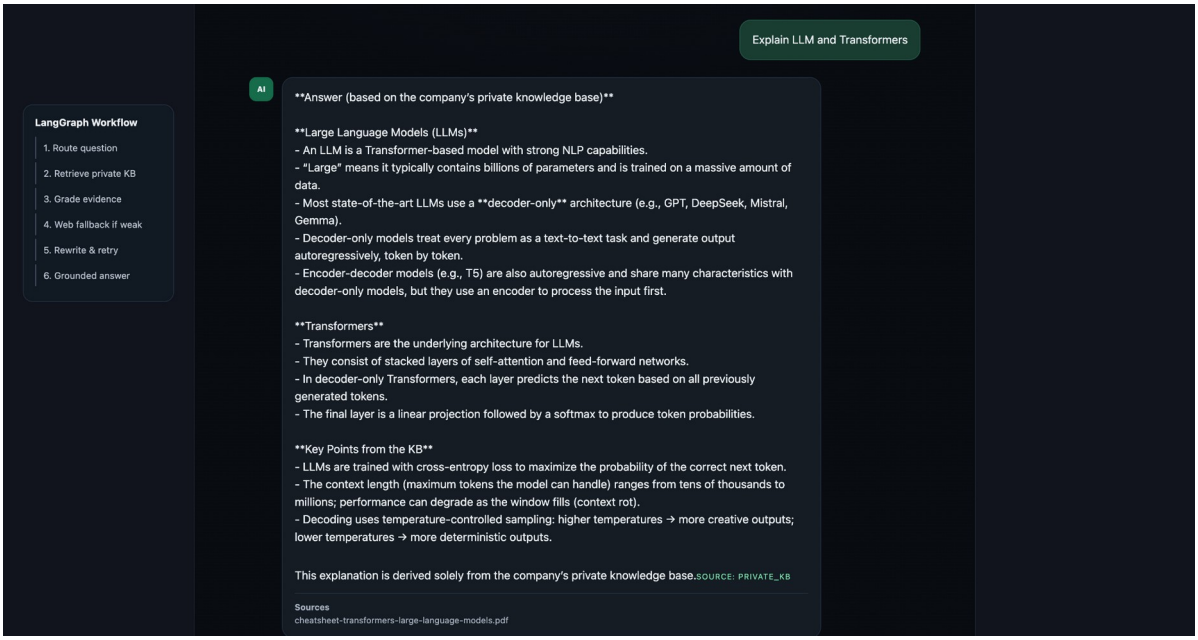


Figure 6. Lightsail image upload reference

9.3 Configure and deploy

Setting	Value
Container name	app
Image	Exact reference returned by push-container-image
Launch command	Blank; Dockerfile command is used
Open port	8080
Protocol	HTTP
Public endpoint	Enabled for container app
Health check path	/api/health

Add environment variables manually in Lightsail. Never upload `.env` or commit secrets to GitHub.

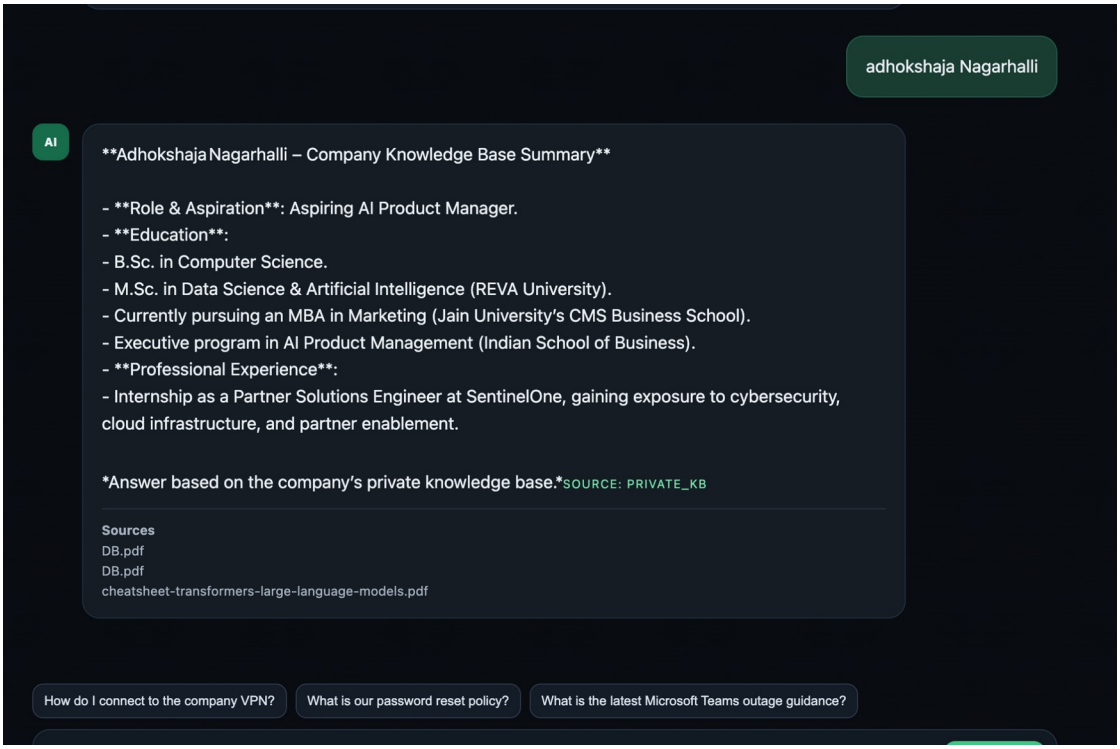


Figure 7. Custom deployment form for the application container

10 Deployment verification

After deployment, wait until the deployment version is marked Active. Open the generated public domain and verify the health endpoint.

```
https://YOUR_PUBLIC_DOMAIN/api/health
https://YOUR_PUBLIC_DOMAIN/docs
```

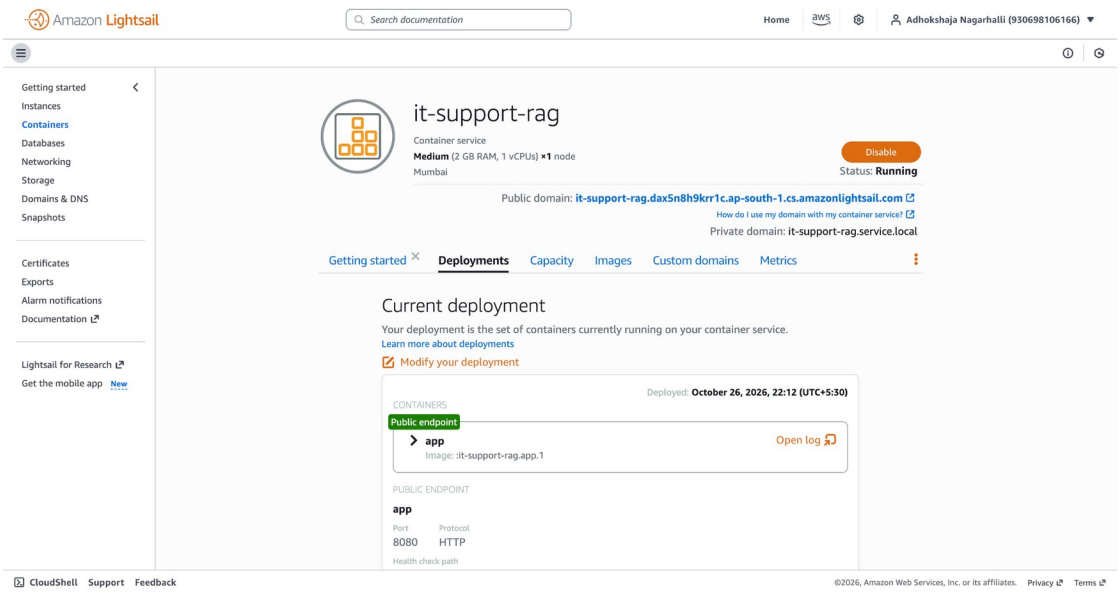


Figure 8. Active Lightsail deployment with public endpoint and health check

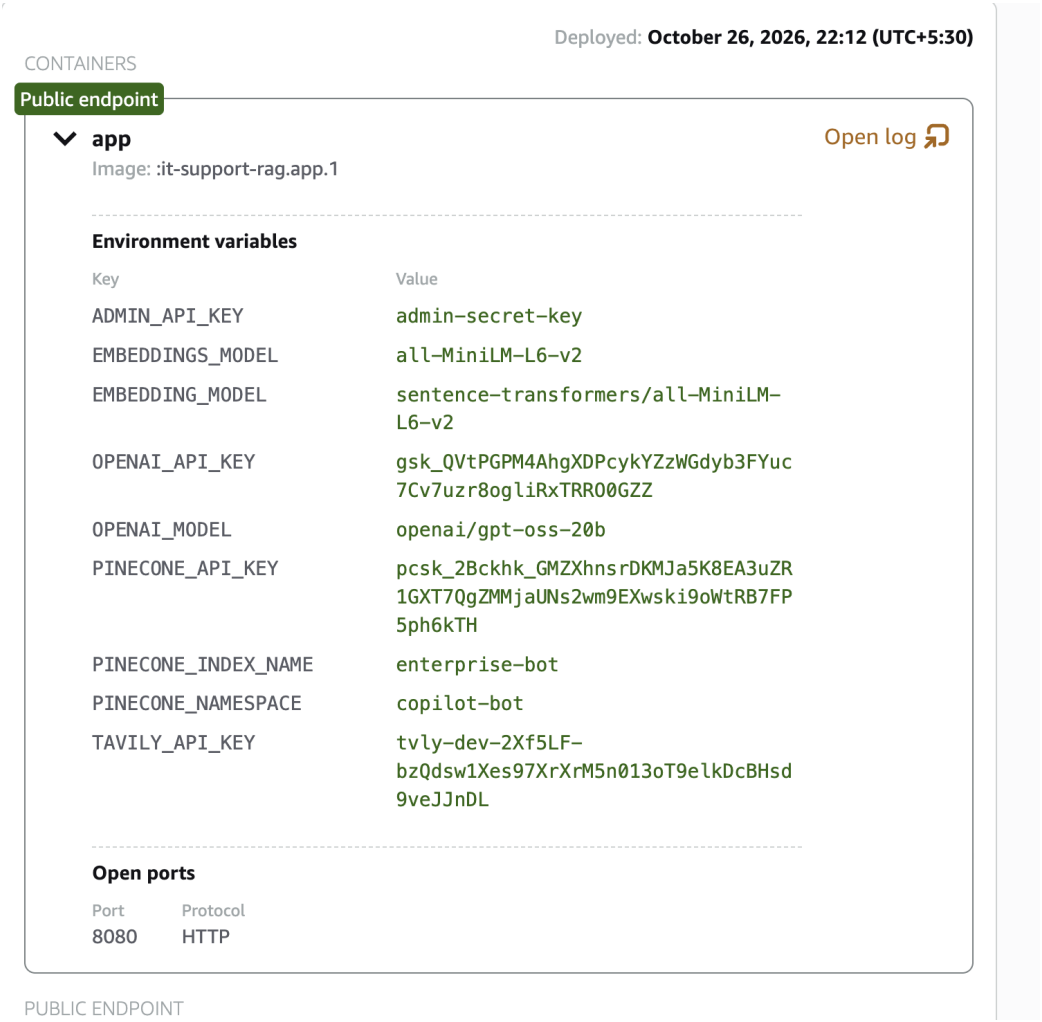


Figure 9. Deployment verification reference

11 Operations and troubleshooting

Symptom	Likely cause	Action
Service registry error	Lightsail service not created or not ready	Create the service, wait for Ready/Running, rerun image push
Deployment unhealthy	Wrong port or health path	Use port 8080 and `/api/health`
Container starts then stops	Missing environment variable or import failure	Open Lightsail logs and verify dependencies and variables
Chat returns provider error	Invalid or missing provider key/model	Check Lightsail environment variables and model name
No private answers	Index, namespace or ingestion mismatch	Confirm Pinecone index, namespace and run ingestion
Image update not visible	Old image reference still deployed	Push with a new label and deploy the new returned reference

12 Security and cost controls

Use environment variables for all secrets. Rotate any key that has been exposed in screenshots, terminals or public repositories. Restrict the admin key and document-ingestion route. For a short demonstration, delete the Lightsail service after testing to prevent ongoing container-service charges.

```
aws lightsail delete-container-service \
  --region ap-south-1 \
  --service-name it-support-rag
```

Deleting the service removes the cloud deployment and public URL. It does not delete the local project or local Docker image.

13 Repeatable deployment checklist

- Project folder contains Dockerfile, requirements.txt and app/
- Docker image builds for linux/amd64
- Local container passes /api/health
- Lightsail service exists and is ready
- Image push returns a Lightsail image reference
- Port 8080 and /api/health are configured
- Environment variables are entered without exposing secrets
- Deployment status becomes Active
- Public /api/health and /docs endpoints work
- Service is deleted after a temporary demo

End of runbook

This document provides the repeatable operating path from local project to verified AWS Lightsail deployment for the Enterprise IT Support Agentic RAG Copilot.